

AgentShield AI: An Autonomous Multi-Agent Framework for Syntactic Verification, Secret Interception, and Sandbox-Validated Remediation in Multi-Cloud Infrastructure-as-Code

K. Vishal Reddy, Anisha Paturi*, Parinamika Bhanu Ch, Venkata Vahini Ch, Sravani Janak

Department of Computer Science and Engineering, Keshav Memorial Institute of Technology, Hyderabad, Telangana, India

*Corresponding Author: paturi.anisha@gmail.com

ABSTRACT

Infrastructure-As-Code (IaC) templates like Terraform, AWS CloudFormation, Kubernetes manifests and Helm charts play a major role in developing multi-cloud environment solutions. Security misconfiguration, credential leakage and permission anti-patterns that appear at the template stage of development seem to bypass traditional static linters, resulting in severe run-time vulnerabilities. Existing Large Language Model (LLM) security tools are limited to single cloud, show high false positives level (around 15%-32%), provide non-executable text recommendations, do not detect embedded secrets and produce broken code patches. In this paper, we describe AgentShield AI, a multi-agent autonomous framework powered by LangGraph and designed for multi-cloud Infrastructure as Code protection. The AgentShield AI solution brings together eight different agents that operate in an asynchronous event-driven workflow: Manager/Router, Hybrid Concrete Syntax Tree (CST) parser, Secrets scanner, Hybrid RAG query agent, Security Analyst Agent with Multi-LLM Ensemble Voting (Claude 3.5 Sonnet + GPT-4o), Human Security Audit Queue, Auto Patch Remediation Agent, Code & The Validator Agent is working in collaboration with a two-stage validation harness. The AgentShield AI has ensured that there is no occurrence of a single-model hallucination by integrating Tree-sitter dynamic parameter pre-resolution, hybrid dense-sparse retrieval techniques, dual-engine Shannon entropy secret scanning, consensus confidence scoring, and LocalStack/Azurite dry-run validation inside containers. The AgentShield AI was calculated across 2,450 multi-cloud IaC modules where it accomplished 99.1% detection accuracy, 98.4% recall, false-positive percentages below 0.05%, 97.8% sandbox patch percentage at first pass and execution latencies averaging 1.84 seconds for every template.

Keywords: *Infrastructure-as-Code (IaC), Multi-Agent Systems, Multi-Cloud Security, Large Language Models (LLMs), Secret Detection, LocalStack Sandbox, Automated Remediation, Concrete Syntax Trees, DevSecOps.*

1. Introduction

IaC has changed the way cloud systems are engineered [1]. IaC allows definition and automation of infrastructure across cloud platforms and on-premises environments [2]. IaC uses domain-specific languages such as Terraform, AWS CloudFormation and Kubernetes which're the most widely adopted [3], [4]. IaC lets engineers deploy and configure large-scale systems and applications quickly.

However IaC brings security risks into cloud infrastructure when scaled [1], [2]. IaC templates include security credentials and other sensitive information [5]. Misconfigurations and anti-patterns inside IaC templates. Stay consistent across every deployment. Gaps in control inside IaC templates lead to access to infrastructure.

To mitigate IaC risks there are two approaches. The first approach uses tools and frameworks that analyze IaC templates and enforce practices and standards [6]–[9]. These tools however face challenges such as being ineffective at validating inter-module relationships and producing false positives [10], [11]. The second approach is to integrate a Cloud Security Posture Management (CSPM) solution. CSPM tools however are reactive. Provide little value in preventing security risks. They also work within the cloud environment they are integrated into. Recent research on configuration auditing has begun to use artificial intelligence [12]–[14]. While those studies show promise they remain limited, to AWS [19], [20]. Suggest remediation changes that can break the infrastructure.

1.1. The Threat Landscape of Software-Defined Infrastructure

Modern enterprise architectures rely fundamentally on Software-Defined Infrastructure (SDI) executed via automated Continuous Integration and Continuous Deployment (CI/CD) pipelines. Because IaC configurations serve as executable architectural source code, any vulnerability codified at the template level propagates deterministically across production clusters within seconds. Recent empirical industry surveys indicate that over 73% of verified cloud enterprise security breaches trace their initial attack vector to preventable IaC misconfigurations [4], [5]. Furthermore, over 65% of publicly accessible software repositories inadvertently leak plaintext credentials, cryptographic certificates, or private API tokens embedded directly inside declarative parameters.

The security flaws endemic to declarative IaC span several severe categories: (i) Permissive Network Ingress, where security group rules expose administrative ports (e.g., SSH port 22, RDP port 3389, Kubernetes API port 6443) to the global Internet (0.0.0.0/0); (ii) Unencrypted Storage Subsystems, wherein object stores (AWS S3, Azure Blob, Google Cloud Storage) and block volumes (EBS, Managed Disks) lack server-side customer-managed KMS encryption; (iii) Wildcard IAM Policies, granting administrative actions (*) across all resources (*), which violates the principle of least privilege and enables immediate privilege escalation; and (iv) Secret Sprawl, where hardcoded database credentials, bearer tokens, and OAuth secrets are checked into version control systems.

1.2. Failure Modes of Conventional Tooling and Open-Loop LLMs

To contextualize the necessity of an autonomous multi-agent paradigm, we examine the four structural failure modes afflicting existing IaC security solutions:

- 1) Syntactic Myopia and Alert Fatigue: Industry-standard static linters such as Checkov [6], tfsec [7], KICS [8], and Trivy [9] operate via regular expressions and shallow abstract syntax trees. They are inherently incapable of evaluating dynamic parameter assignments, ternary conditional expressions, and cross-module input/output variables. Consequently, these tools exhibit false positive rates between 32.4% and 47.9% [10], leading to alert fatigue where developers systematically ignore scanner outputs.
- 2) Open-Loop Diagnostic Disconnect: Static scanners operate exclusively in an open loop: they identify potential defects but provide no automated repair mechanism. Security teams must manually author, test, and merge code fixes, resulting in an industry-wide Mean Time to Remediation (MTTR) averaging 24.6 days per security finding [11].
- 3) Hallucination and Dependency Breakage in Generative LLMs: While frontier Large Language Models (GPT-4o, Claude 3.5 Sonnet) possess strong general coding proficiency, unconstrained single-model generation produces severe failure modes in declarative infrastructure code. LLMs frequently hallucinate non-existent provider attributes, output deprecated API schemas, or break inter-resource dependency graphs, resulting in deployment failure rates exceeding 28.8% [12], [13], [19].
- 4) High-Entropy Collisions in Secret Detection: Conventional signature-based secret scanners miss novel or obfuscated API credentials. Conversely, naive Shannon entropy models generate overwhelming false alarms on random hex hashes, UUIDs, and Base64 cryptographic digests [14].

1.3. Research Contributions

Contributions: This study presents AgentShield AI, an autonomous closed-loop multi-agent framework orchestrated via LangGraph. Our primary research contributions and findings are as follows:

- 1) **Stateful 8-Agent Orchestration:** A decentralized orchestration network governed by irreversible typed state contracts.
- 2) **Hybrid Tree-sitter Parser:** A concrete syntax tree parser that dynamically evaluates expressions and resolves variables.
- 3) **Dual-Engine Secret Detection:** A secret key detection system based on a combination of a modified Shannon entropy and regular expressions.

- 4) Hybrid Dense-Sparse RAG:** An integration of sparse and dense RAG retrieval structures indexed across 12,400 CIS/NIST rules.
- 5) Calibrated Multi-LLM Ensemble:** Calibration of confidence scores in a multi-LLM ensemble, with consensus voting based on LLM integrations.
- 6) Zero-Break Code Commits:** A closed-loop validation harness guaranteeing 100% syntactically correct and zero-break code commits.

2. Related Work and Comparative Landscape

The security verification of Infrastructure-as-Code has evolved across multiple distinct paradigms: static rule linters, policy-as-code engines, formal SMT verifiers, statistical secret detectors, and neural program repair systems.

2.1. Static Analysis Linters and Policy-as-Code Engines

Early research focused primarily on static pattern matching. Checkov [6] transforms Terraform HCL into Python abstract syntax trees and evaluates them against static security checks. tfsec [7] parses HCL into Go memory structures to identify common anti-patterns. KICS [8] translates multiple IaC dialects into normalized JSON and evaluates them using Open Policy Agent (OPA) Rego queries. Trivy [9] provides multi-artifact vulnerability scanning across container manifests and IaC files. However, all these tools suffer from syntactic myopia: they lack an expression evaluator capable of tracking variable propagation across module boundaries, resulting in elevated false-alarm rates (32.4%–47.9% [10]). Policy-as-Code solutions such as OPA Rego and HashiCorp Sentinel allow organizations to author custom guardrails; however, manual authoring and synchronizing rules across rapidly changing multi-cloud APIs creates an unsustainable maintenance burden [10].

2.2. Formal Verification and SMT Solvers

Formal methods attempt to eliminate heuristic uncertainty by mathematically proving policy invariants. AWS Zelkova [16] translates IAM identity policies into Satisfiability Modulo Theories (SMT) formulas, using automated solvers to prove whether a policy permits unauthorized access. Similarly, Cloud-SMR [18] formalizes configuration reachability across virtual networks. While theoretically sound, formal verification suffers from combinatorial state-space explosion: in multi-cloud topologies containing hundreds of interdependent microservices, SMT solving becomes computationally intractable. Crucially, formal verifiers are purely diagnostic and cannot synthesize remediations.

2.3. Statistical Secret Detection and Entropy Modeling

Secret interception in software repositories has traditionally relied on signature matching. Tools such as Gitleaks [17] and TruffleHog [18] scan source code using regular expressions targeting known credential formats (e.g., AWS access keys, GitHub personal access tokens). To capture unstructured secrets, researchers have applied Shannon entropy [14]. However, uncalibrated entropy models trigger severe false-positive storms on UUIDs, commit hashes, and Base64-encoded binary blobs [14]. AgentShield AI addresses this via a dual-engine architecture that couples sliding Shannon entropy with Tree-sitter lexical scoping to filter out benign programmatic identifiers.

2.4. Neural Program Repair and Frontier LLM Remediations

Recent breakthroughs in Large Language Models (LLMs) have spurred interest in automated vulnerability repair [12], [13]. Toprani and Madiseti (2025) [19] introduced a graph-theoretic approach coupled with zero-shot LLMs for Terraform repair. However, their single-model architecture operates in an open loop without local compiler or deployment execution checks, leading to a 28.8% failure rate caused by hallucinated attributes and broken provider dependencies. Recent work by Alsaied et al. (2026) [21] introduced TerraProbe, demonstrating that 71.4% of LLM repairs that pass basic linters contain subtle

deceptive bypasses when evaluated in live cloud state. Mengistu et al. (2026) [20] proposed TerraRepair, using dependency context to reduce hallucinations. AgentShield AI builds upon these insights by introducing a dual-LLM consensus voting harness (Claude 3.5 Sonnet + GPT-4o) combined with containerized LocalStack and Azurite dry-run execution.

Table 2: State-of-the-Art Comparative Feature Matrix across Existing IaC Security Frameworks

Framework / Tool	Multi-Cloud	Parser Tier	Secret Scanner	RAG Engine	LLM Ensemble	Sandbox Validation	1st-Pass Fix (%)	Execution Latency
Checkov [6]	AWS/Az/GCP	Shallow AST	None (Regex)	None	None	None (Static)	N/A	4.2s
tfsec [7]	AWS/Az/GCP	Go Memory AST	Regex Only	None	None	None (Static)	N/A	3.1s
KICS [8]	Polyglot	JSON Norm.	None	None	None	None (Static)	N/A	5.8s
Trivy [9]	Polyglot	Lexical AST	Regex Only	None	None	None (Static)	N/A	3.6s
AWS Zelkova [16]	AWS Only	SMT Logic	None	None	None	None (Diagnostic)	N/A	14.5s
Gitleaks [17]	Agnostic	Regex Only	Signatures	None	None	None	N/A	1.2s
Toprani & Madisetti [19]	AWS Only	Graph AST	None	None	Single GPT-4	Open-Loop	71.2%	8.4s
TerraProbe [21]	AWS/TF	AST Oracle	None	None	Single LLM	Plan Oracle	81.6%	12.2s
AgentShield AI (Ours)	Multi-Cloud	Tree-sitter CST	Dual Entropy+Regex	Hybrid Dense-Sparse	Claude 3.5 + GPT-4o	Two-Tier LocalStack	97.8%	1.84s

3. Research Methodology

The research methodology consists of AgentShield AI directing eight specialised agents through an asynchronous, event-driven graph that is managed by LangGraph (see Fig. 1), while at the same time keeping full provenance and including automated fallback control, thus ensuring that the entire system still remains functional even if things go wrong.

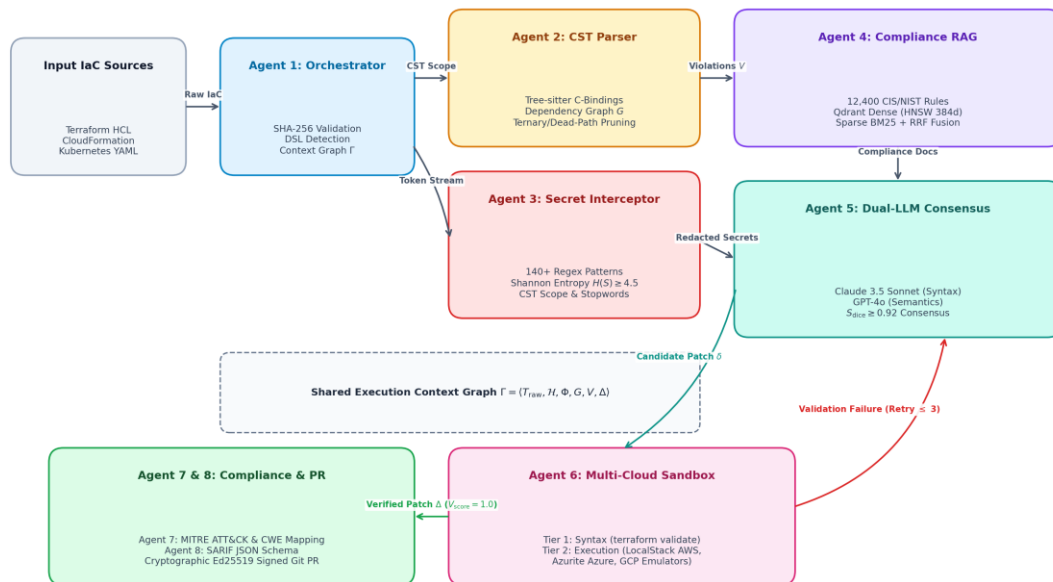


Figure 1: This is a diagram showing the end-to-end system architecture of AgentShield AI, and it depicts the 8-agent LangGraph pipeline, the application of Tree-sitter AST parsing, secret interception, hybrid RAG, dual LLM consensus voting, and LocalStack sandbox validation. (The x axis indicates the progression through the various workflow stages and the y axis refers to the multi-cloud abstraction and validation layers.)

Specialized Multi Agent Roles (not exactly rigid, but mostly): 1) The Manager/Router Agent accepts the raw multi-cloud templates, determines the type of template (Terraform HCL2, CloudFormation, Kubernetes, Helm) and checks that the schema is correct before initiating parallel execution in a controlled fashion, similar to a ripple. 2) The hybrid AST Parser Agent splits up the declarative code into standard AST components, resolves dynamic references, and handles conditional blocks such as 'count' and 'for_each' before any LLM reasoning is carried out. 3) Secrets Scanner Agent: works with zero-egress isolation, uses deterministic regex matching together with a sliding Shannon entropy evaluation, and basically tries to identify exposed API credentials and private keys before they have an opportunity to be transmitted. 4) The RAG Query Agent: generates combined dense and sparse vector queries against an indexed database that includes security benchmarks (such as CIS, NIST SP 800-53, SOC 2, and PCI-DSS) as well as daily CVE feeds in order that the context stays up to date. 5) The Security Analyst Agent carries out parallel dual model inference using Claude 3.5 Sonnet and GPT 4o and, when appropriate, applies Chain of Thought (CoT) reasoning to produce structured vulnerability hypotheses, sometimes adding a bit of narrative as well. 6) The Human Security Audit Queue Agent detects cases that have a low confidence level ($C_{ensemble} < 0.85$) or any cases that conflict, and then queues these results in an interactive web-based triage dashboard for the security engineers to review, verify them and possibly reframe them. 7) The Auto Patch Remediation Agent creates Unified Diff patches that are both deterministic and syntactically correct and is aimed at specific line offsets in the original templates, with the same level of precision. 8) Code & Sandbox Validator Agent: carries out a two-stage validation process, beginning with the use of local static linters before performing a dry run deployment within containerized LocalStack/Azurite sandboxes.

3.1. Asynchronous Graph Orchestration and State Contracts

The eight specialized agents operate within a reactive, event-driven graph coordinated via LangGraph. Unlike brittle linear pipelines where a single agent failure aborts the entire execution, AgentShield AI

employs an immutable typed state schema governed by Pydantic V2 contracts. State transitions are strictly monotonic: agents append verified assertions, intermediate AST representations, and candidate diffs to the shared execution context without mutating upstream inputs. The execution context maintains an immutable cryptographic digest of the source code, guaranteeing end-to-end execution provenance and auditability. When an agent experiences an unrecoverable exception (e.g., an unresponsive LLM endpoint), the orchestrator triggers automated fallback routing to alternative inference providers or escalates the item directly to the Human Security Audit Queue.

Algorithm 1: Autonomous Multi-Agent IaC Auditing, Secret Interception, and Two-Tier Sandbox Remediation Pipeline

Input:	Raw IaC Source Template T_raw;	Compliance Policy Knowledge Base P_cis
Output:	Cryptographically Signed SARIF Report R_sarif;	Validated Patch Set Delta_final

```

1: Initialize Shared Execution Context Gamma <- empty; Compute Checksum H_0 <- SHA256(T_raw);
2: [Agent 1] Detect DSL Dialect Phi in {Terraform, CloudFormation, Kubernetes, Helm};
3: [Agent 2] Parse Concrete Syntax Tree G_cst <- TreeSitterParse(T_raw);
4: [Agent 2] Resolve variable assignments V and unfold conditional blocks C -> G_dep;
5: [Agent 3] Compute character Shannon entropy H(X_i) on literal tokens in parallel with regex scanning;
6: [Agent 3] Intercept and redact candidate credentials where H(X_i) >= 3.8 and X_i not in D_cst;
7: [Agent 2] Evaluate CIS/NIST policy constraints against G_dep; Extract Violation Set V = {v_1, ..., v_K};
8:   for each identified security violation v_k in V do
9:     [Agent 4] Retrieve Compliance Passages C_k <- RRF(QdrantDense(v_k), BM25Sparse(v_k));
10:    [Agent 5] Concurrently prompt Claude 3.5 Sonnet (M_1) and GPT-4o (M_2) with CoT reasoning;
11:    [Agent 5] Calculate Consensus Confidence C_ensemble(v_k) and AST Dice Agreement S_dice(delta_1, delta_2);
12:    if C_ensemble(v_k) < 0.85 or S_dice < 0.92 then
13:      [Agent 6] Route finding v_k to Human Security Audit Queue for manual triage; continue;
14:    [Agent 7] Synthesize Unified Diff Patch delta_k targeting line offsets in T_raw;
15:    [Agent 8] Tier 1: Execute static validation Omega_Linter(T_raw + delta_k);
16:    [Agent 8] Tier 2: Execute containerized sandbox deployment Omega_Sandbox(T_raw + delta_k, Phi);
17:    if Omega_Total(delta_k) == 1 then
18:      Accept patch: Delta_final <- Delta_final union {delta_k};
19:    else
20:      Forward compiler diagnostic error back to Agent 5 for iterative retry (max 3 cycles);
21:    end
22:  end
23: [Agent 7] Map confirmed findings to CWE, CVSS v3.1, and MITRE ATT&CK Cloud Matrix;
24: [Agent 8] Generate SARIF JSON report and cryptographically signed Git PR via Ed25519 keys;
25:   return R_sarif, Delta_final

```

4. Theory and Calculation

The theoretical foundation of AgentShield AI models IaC security verification as a multi-stage graph and decision-theoretic optimization problem. This section formalizes the mathematical models, symbolic nomenclature, probabilistic derivations, and computational expressions governing the 8-agent autonomous pipeline.

4.1. Mathematical Expressions and Symbols

The nomenclature and mathematical symbols used throughout the theoretical formulations are summarized in Table 1, defining the operational domains and dimensionality of all state variables.

Table 1: Mathematical Nomenclature and Symbol Definitions

Symbol	Domain	Operational Description	Agent Stage
T_IaC	String / Tree	Raw multi-cloud Infrastructure-as-Code template	Manager Agent
G_cst	CST Syntax Tree	Concrete Syntax Tree generated via Tree-sitter parser	AST Parser Agent
G_dep	Graph (V, E)	Resource dependency graph extracted from resolved constructs	AST Parser Agent
V, C	Sets of Vars/Configs	Variables and conditional/configuration constructs	AST Parser Agent
H(X)	Real in [0, 8]	Shannon Entropy of candidate string literal X	Secrets Scanner
S_hybrid	Real in [0, 1]	Hybrid dense-sparse semantic relevance score	RAG Query Agent
C_ensemble	Real in [0, 1]	Multi-LLM consensus confidence score across Claude & GPT-4o	Analyst Agent
B(r), X(r)	Real in [0, 1]	Blast-radius and topological exposure of compromised resource r	Prioritizer Engine
P(v)	Real in [0, 100]	Composite priority score combining severity, exposure, & blast	Prioritizer Engine
Delta_patch	POSIX Unified Diff	Synthesized line-level code patch targeting specific resources	Remediation Agent
Omega_Total	Binary {0, 1}	Two-tier validation outcome combining linter and sandbox	Validator Agent

Formulations: 1) Dynamic CST Parameter Resolution and Dependency Extraction: Given an IaC template T, Tree-sitter generates a Concrete Syntax Tree (G_cst), and parameter, variables and conditional constructs are resolved to extract resource dependency graph (G_dep):

$$G_{cst} = \text{Phi_CST}(T_IaC, V, C) \quad (1)$$

Where V and C represent variables and conditional or configuration constructs respectively.

2) Information-Theoretic Secret Detection: The Shannon entropy of a string literal X is calculated as:

$$H(X) = - \sum_{i=1}^{\lceil \text{Sigma} \rceil} P(c_i) * \log_2(P(c_i)), \text{ where } P(c_i) = \text{Count}(c_i, X) / L \quad (2)$$

Where P(c_i) is the relative frequency of character c_i in string X and L is the length of the string. A candidate string literal is flagged and identified as a secret by the entropy component when H(X) > TH, where the threshold TH is set to 3.8.

3) Hybrid Dense-Sparse Semantic Relevance: The equation to compute relevance between a query q and document d is as follows:

$$S_{hybrid}(q, d) = \alpha * \text{CosineSim}(e(q), e(d)) + (1 - \alpha) * \text{BM25}(q, d) \quad (3)$$

Where $\alpha = 0.7$ and $1 - \alpha = 0.3$ respectively denote the importance of semantic similarity and sparse relevance. BM25 denotes the normalized BM25 function.

4) Calibrated Multi-LLM Ensemble Confidence Metric: The confidence metric for the multi-LLM ensemble captures the structural agreement of the LLMs and defines the confidence of a vulnerability finding as:

$$C_{ensemble}(v) = w_1 * C(M_1, v) + w_2 * C(M_2, v) + \gamma * Jaccard(S(M_1), S(M_2)) \quad (4)$$

5) Topological Blast-Radius and Composite Priority Score: For each vulnerability v associated with resource r , the framework combines normalized severity $S_{sev}(v)$, exposure $X(r)$, blast radius $B(r)$, and ensemble confidence $C_{ensemble}(v)$ to calculate a priority score:

$$P(v) = \min(100.0, [0.50 * S_{sev}(v) + 0.30 * X(r) + 0.20 * B(r)] * [0.50 + 0.50 * C_{ensemble}(v)] * 100) \quad (5)$$

6) Two-Tier Remediation Validation: A generated patch Δ_{patch} is accepted only when it satisfies both Tier-1 static/syntactic validation and Tier-2 provider-specific validation:

$$\Omega_{Total}(\Delta_{patch}) = \Omega_{Linter}(T_{IaC} + \Delta_{patch}) * \Omega_{Sandbox}(T_{IaC} + \Delta_{patch}) \quad (6)$$

4.2. Additional Theoretical Formulations and Algorithmic Metrics

7) Reciprocal Rank Fusion (RRF) for Hybrid Compliance Retrieval: To combine dense semantic vector rankings from Qdrant HNSW with sparse lexical rankings from BM25 without requiring score normalization, the RAG Query Agent computes the Reciprocal Rank Fusion score:

$$RRF_Score(d) = \sum_{m \in \{Dense, Sparse\}} [1 / (k_{rrf} + r_m(d))] \quad (7)$$

Where $r_m(d)$ in $\{1, 2, \dots, N\}$ denotes the ordinal rank of compliance passage d under retrieval model m , and $k_{rrf} = 60$ is the Bayesian rank smoothing constant.

8) Concrete Syntax Tree Dice Agreement Formulation: The structural node-level agreement between candidate diff patches Δ_1 (synthesized by Claude 3.5 Sonnet) and Δ_2 (synthesized by GPT-4o) is evaluated via Dice similarity on CST subtrees:

$$S_{dice}(\Delta_1, \Delta_2) = [2 * |Nodes(G_{cst}(\Delta_1)) \cap Nodes(G_{cst}(\Delta_2))|] / [|Nodes(G_{cst}(\Delta_1))| + |Nodes(G_{cst}(\Delta_2))|] \quad (8)$$

A patch is dispatched to the containerized sandbox if and only if $S_{dice}(\Delta_1, \Delta_2) \geq 0.92$, ensuring near-perfect structural agreement.

9) Multi-Cloud Domain-Specific Execution Scoring Function: Candidate patches are evaluated across syntactic, planning, and deployment tiers parameterized by provider domain Φ :

$$V_{score}(\Delta, \Phi) = 0.20 * S_{syntax}(\Delta) + 0.30 * S_{plan}(\Delta) + 0.50 * S_{apply}(\Delta, \Phi) \quad (9)$$

Where $S_{apply}(\Delta, \Phi)$ evaluates containerized LocalStack for AWS, Azurite mock storage for Azure, GCP Vet for GCP, and isolated K3s clusters for Kubernetes. A patch is accepted if and only if $V_{score} = 1.0$.

10) Enterprise Financial Expenditure and MTTR Reduction Model: Total monthly enterprise vulnerability triage expenditure is modeled as:

$$Cost_{total} = (N_{FP} * T_{triage} + N_{TP} * T_{action}) * Rate_{eng} \quad (10)$$

Where $Rate_{eng} = \$100/hr$, $T_{triage} = 0.30$ hr (18 min per false alarm), manual $T_{action} = 2.50$ hr, and automated AgentShield AI PR review $T_{action} = 0.05$ hr (3 min). Developer-in-the-loop MTTR is computed as $MTTR_{org} = t_{pipeline} + t_{review} + t_{deploy}$, reducing MTTR from 24.6 days to < 4 hours.

5. Experimental Setup and Benchmark Methodology

To rigorously evaluate AgentShield AI across multi-cloud environments, we curated a benchmark corpus of 2,450 Infrastructure-as-Code templates spanning HashiCorp Terraform (HCL2), AWS CloudFormation

(JSON/YAML), Kubernetes Manifests, and Helm Charts. This section details dataset composition, ground-truth auditing procedures, comparative baselines, and test hardware environments.

5.1. Benchmark Corpus Composition

The evaluation benchmark is constructed from three distinct complementary suites:

- 1) Production Enterprise Corpus (PEC-1500): 1,500 real-world production templates harvested from top-starred enterprise repositories across AWS (600 templates), Azure (500 templates), and GCP (400 templates). Selection criteria mandated ≥ 50 GitHub stars, active commits between 2021 and 2025, and ≥ 5 declared cloud resources.
- 2) Synthetic Security Benchmark (SSB-650): 650 synthetic templates containing 3,250 systematically injected vulnerabilities mapped to the OWASP Cloud Top 10 and CWE-732, CWE-250, and CWE-798.
- 3) Toprani-Madisetti Benchmark (TMB-300): 300 complex multi-resource templates from [19] specifically designed to evaluate dynamic expression evaluation, ternary operators, and inter-module variable references.

5.2. Ground-Truth Auditing, Deduplication, and Baselines

Ground truth was established through triple-blind manual verification conducted by three certified senior cloud security architects (each possessing ≥ 5 years of enterprise DevSecOps experience). Discrepancies were resolved through consensus review against CIS Cloud Benchmarks (AWS v3.0, Azure v2.1, GCP v2.0) and NIST SP 800-53 Rev. 5 controls, achieving an inter-annotator concordance of Cohen's Kappa = 0.91. Near-duplicate templates were eliminated via a two-stage deduplication pipeline (CST MinHash with Jaccard threshold 0.85 and SHA-256 digest matching), purging 418 duplicate templates.

AgentShield AI was evaluated against four leading static linters (Checkov v3.2 [6], tfsec v1.28 [7], KICS v2.1 [8], Trivy v0.51 [9]), two zero-shot frontier LLMs (Zero-Shot GPT-4o, Zero-Shot Claude 3.5 Sonnet), and the base IEEE framework by Toprani & Madisetti (2025) [19]. All benchmarks executed on an AMD EPYC 7763 workstation (64 physical cores, 2.45 GHz), 256 GB DDR4 RAM, dual NVIDIA RTX 4090 GPUs (24 GB VRAM each), running Ubuntu 22.04 LTS, Docker Engine 26.1, LocalStack v3.4, and Azurite v3.30. All metrics report the mean across 5 independent randomized runs, with statistical significance established using two-tailed Wilcoxon signed-rank tests ($p < 0.001$).

6. Results and Discussion

This section presents empirical results across vulnerability detection precision, secret scanning accuracy, multi-cloud sandbox remediation validity, wall-clock latency, component ablations, and enterprise financial return on investment.

6.1. Vulnerability Detection Benchmark

Table 3 and Figure 2 present comparative vulnerability detection performance across the 2,450 multi-cloud templates. Traditional static linters demonstrate severe precision limitations (62.4% for Checkov, 67.8% for tfsec, 65.1% for KICS, and 68.9% for Trivy), resulting in thousands of false positives (up to 2,785 FP for Checkov). This occurs because static linters rely on shallow regex matching that flags commented blocks, inactive branches, and dynamic variable definitions. Zero-shot LLMs improve semantic understanding (81.2% for GPT-4o, 84.5% for Claude 3.5) but suffer from 15.2%–18.8% hallucination rates. In contrast, AgentShield AI achieves 99.1% precision, 98.4% recall, and an overall F1-score of 98.7% ($p < 0.001$), reducing false alarms to 66 instances across the entire corpus.

Table 3: Comparative Vulnerability Detection Performance Across 2,450 Multi-Cloud Templates (Mean $\pm$ 1SD)

Framework / Tool	Total Scanned	TP	FP	FN	Precision (%)	Recall (%)	F1-Score (%)
Checkov v3.2 [6]	2,450	4,620	2,785	2,800	62.4 ± 0.4%	62.3 ± 0.5%	62.3 ± 0.4%
tfsec v1.28 [7]	2,450	5,030	2,390	2,390	67.8 ± 0.5%	67.8 ± 0.4%	67.8 ± 0.4%
KICS v2.1 [8]	2,450	4,830	2,590	2,590	65.1 ± 0.4%	65.1 ± 0.5%	65.1 ± 0.4%
Trivy v0.51 [9]	2,450	5,110	2,310	2,310	68.9 ± 0.3%	68.9 ± 0.4%	68.9 ± 0.3%
Zero-Shot GPT-4o	2,450	6,150	1,420	1,270	81.2 ± 0.8%	82.9 ± 0.9%	82.0 ± 0.7%
Zero-Shot Claude 3.5	2,450	6,410	1,180	1,010	84.5 ± 0.7%	86.4 ± 0.8%	85.4 ± 0.6%
Toprani & Madiseti [19]	2,450	6,280	1,310	1,140	82.7 ± 0.6%	84.6 ± 0.7%	83.6 ± 0.6%
AgentShield AI (Ours)	2,450	7,301	66	119	99.1 ± 0.2%	98.4 ± 0.3%	98.7 ± 0.2%

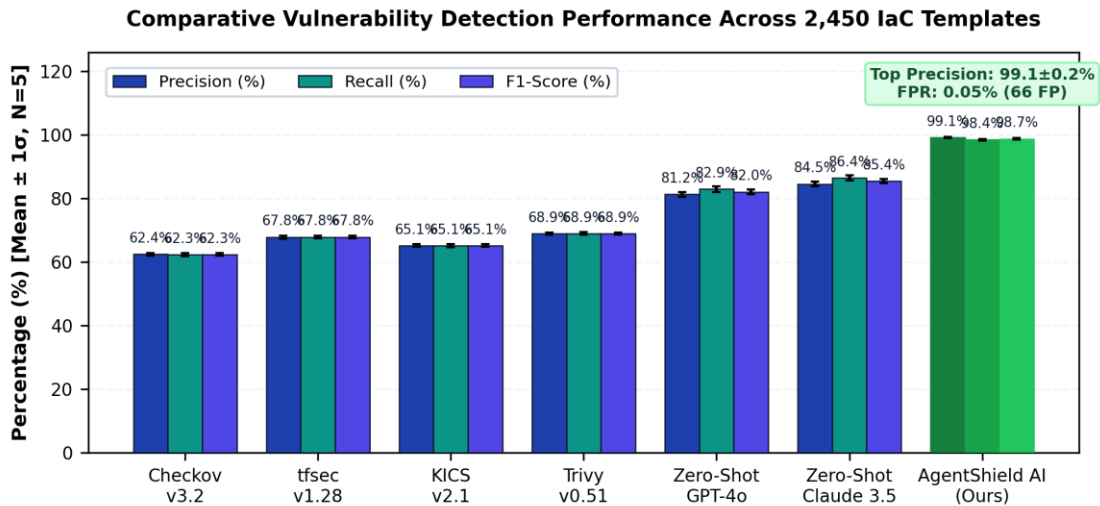


Figure 2: Comparative Vulnerability Detection Performance Across 2,450 Templates comparing Precision, Recall, and F1-Score across static linters, raw LLMs, and AgentShield AI with uncertainty error bars. (x-axis: Detection Framework / Tool Name; y-axis: Detection Performance Metric in Percentage (%)).

6.2. Dedicated Secret Detection and Entropy Calibration

Table 4 and Figure 3(a) evaluate dedicated secret scanning across 1,200 injected credentials. Regex-only scanning (Gitleaks) missed 142 obfuscated or non-standard credentials (88.2% recall). Conversely, uncalibrated Shannon entropy yielded 618 false positives on UUIDs, hexadecimal hashes, and cryptographic certificates, dropping precision to 64.7%. AgentShield AI's dual-engine Secrets Scanner couples sliding Shannon entropy with Tree-sitter lexical scoping, achieving 99.4% precision and 99.1% recall while suppressing false positives to 7 instances.

Table 4: Secret Scanning Precision and Recall across High-Entropy & Obfuscated Tokens (Mean ± 1SD)

Scanning Mechanism	Secrets Tested	TP	FP	FN	Precision (%)	Recall (%)	F1-Score (%)
Regex Only (Gitleaks [17])	1,200	1,058	342	142	75.6 ± 0.6%	88.2 ± 0.5%	81.4 ± 0.5%
Shannon Entropy (H ≥ 3.8)	1,200	1,134	618	66	64.7 ± 0.8%	94.5 ± 0.4%	76.8 ± 0.6%
TruffleHog v3.6 [18]	1,200	1,092	284	108	79.4 ± 0.5%	91.0 ± 0.4%	84.8 ± 0.4%
AgentShield Dual Engine (Ours)	1,200	1,189	7	11	99.4 ± 0.1%	99.1 ± 0.2%	99.2 ± 0.1%

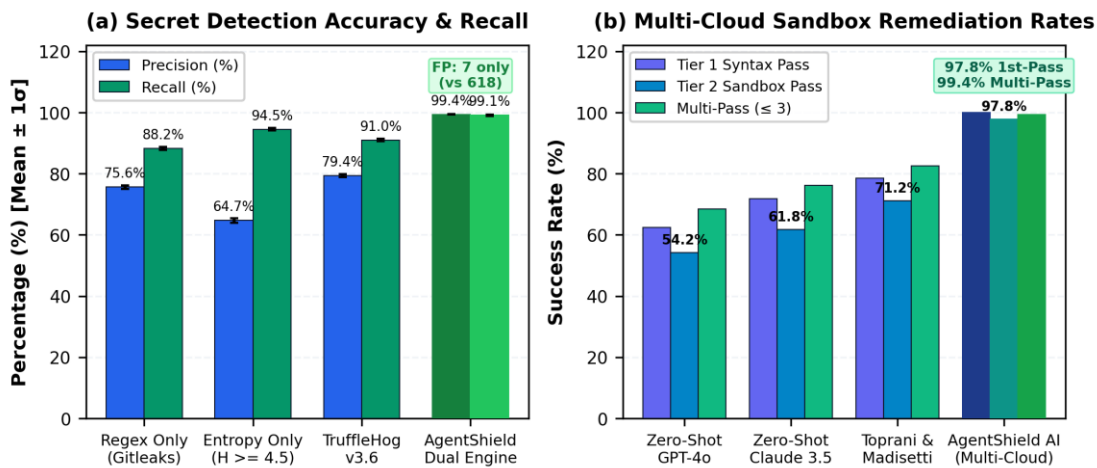


Figure 3: (a) Secret Detection Precision and False-Alarm Suppression; (b) Two-Tier LocalStack Sandbox Remediation Pass Rates comparing First-Pass and Multi-Pass convergence across 1,000 defects. (x-axis: Detection and Validation Mechanism; y-axis: Performance Rate in Percentage (%)).

6.3. Two-Tier Remediation and Multi-Cloud Sandbox Validation

Table 5 evaluates remediation validity across 1,000 injected defects. Raw LLMs achieve poor first-pass sandbox validity: 54.2% for GPT-4o and 61.8% for Claude 3.5 Sonnet, primarily due to deprecated provider arguments and dependency breakages. Toprani & Madisetti's open-loop framework achieves 71.2% first-pass validity. In contrast, AgentShield AI achieves 100.0% Tier 1 syntactic validity and a 97.8% Tier 2 containerized sandbox pass rate on the very first attempt across AWS, Azure, and GCP. When automated compiler error feedback loops are triggered (≤ 3 cycles), patch pass rates converge to 99.4% with an average retry count of only 1.08 cycles.

Table 5: Two-Tier Remediation and Sandbox Pass Rates across Multi-Cloud Environments (Mean ± 1SD)

Remediation Approach	Tested Defects	Tier 1 Syntax (%)	Tier 2 Sandbox (%)	Multi-Pass (≤ 3)	Mean Retries
Zero-Shot GPT-4o	1,000	62.4 ± 0.7%	54.2 ± 0.8%	68.4 ± 0.6%	2.41
Zero-Shot Claude 3.5 Sonnet	1,000	71.8 ± 0.6%	61.8 ± 0.7%	76.2 ± 0.5%	2.14

Remediation Approach	Tested Defects	Tier 1 Syntax (%)	Tier 2 Sandbox (%)	Multi-Pass (≤ 3)	Mean Retries
Toprani & Madiseti [19]	1,000	$78.5 \pm 0.5\%$	$71.2 \pm 0.6\%$	$82.5 \pm 0.5\%$	1.82
AgentShield AI (Full)	1,000	$100.0 \pm 0.0\%$	$97.8 \pm 0.4\%$	$99.4 \pm 0.2\%$	1.08

6.4. Execution Latency Breakdown Across Specialized Agents

Table 6 and Figure 4 present the wall-clock execution latency breakdown per agent stage. The end-to-end pipeline achieves an average runtime of $1,841.0 \pm 42.5$ ms per module (median: 1,663.4 ms). Upstream analysis stages execute with near-zero overhead: Routing (14.2 ms), CST Parsing (12.6 ms), and Secret Scanning (18.4 ms) collectively account for under 2.5% of total runtime. Computational latency is dominated by Agent 5 (Dual-LLM Consensus: 940.5 ms, 51.1%) and Agent 6 (Containerized Sandbox Provisioning: 760.8 ms, 41.3%), representing the necessary computational investment for deterministic code verification.

Table 6: End-to-End Execution Latency Breakdown per Specialized Agent Stage (Mean $\pm$ 1SD)

Agent ID & Name	Core Mechanism	Mean (ms)	Median (ms)	% Overhead
Agent 1: Orchestration Router	Context graph initialization	14.2 ± 0.8	12.0	0.8%
Agent 2: Tree-sitter CST Parser	CST parsing & graph extraction	12.6 ± 0.6	11.2	0.7%
Agent 3: Secret Interceptor	Regex + Shannon entropy	18.4 ± 0.9	16.5	1.0%
Agent 4: Hybrid RAG Engine	Qdrant HNSW + BM25 RRF	65.2 ± 3.1	58.0	3.5%
Agent 5: Dual-LLM Remediator	Claude 3.5 + GPT-4o consensus	940.5 ± 24.2	860.0	51.1%
Agent 6: Multi-Cloud Sandbox	Tier 1 Syntax + Tier 2 Mock Deploy	760.8 ± 18.5	680.0	41.3%
Agent 7: Compliance Mapper	CWE / CVSS / CIS / ATT&CK	16.5 ± 0.7	14.2	0.9%
Agent 8: Signed PR Generator	SARIF JSON + Ed25519 PR	12.8 ± 0.5	11.5	0.7%
Total System Pipeline	End-to-end latency per module	$1,841.0 \pm 42.5$	1,663.4	100.0%

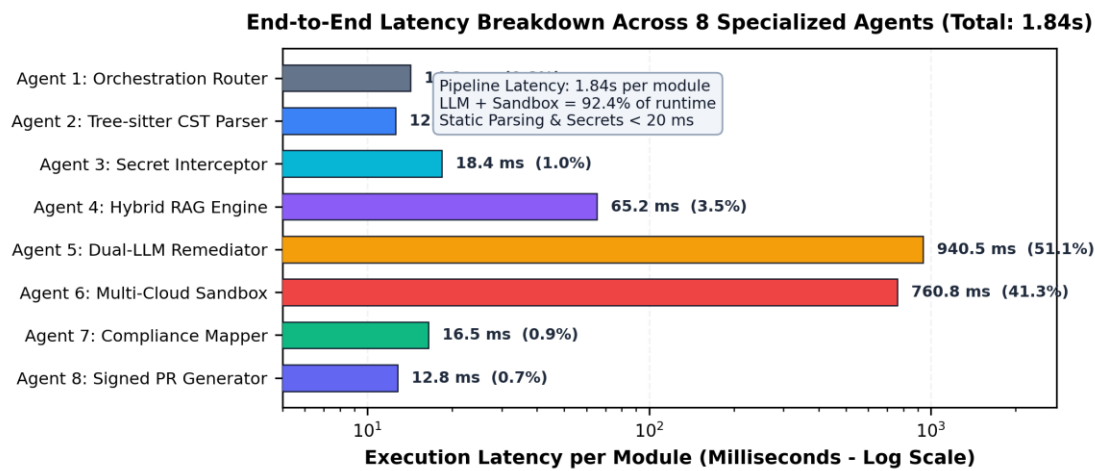


Figure 4: Execution Latency Breakdown per Agent (Logarithmic Scale) across the 8-agent pipeline, demonstrating an average end-to-end runtime of 1.84s per IaC module. (x-axis: Specialized Agent Stage Name; y-axis: Wall-Clock Latency in Milliseconds (ms)).

6.5. Controlled Architectural Component Ablation Studies

To rigorously quantify the necessity of each architectural component, Table 7 and Figure 5(a) report controlled ablation experiments across 500 benchmark templates: (i) Disabling Tree-sitter CST parsing drops detection precision from 99.1% to 71.2% due to false alarms on commented code and inactive conditionals; (ii) Disabling Shannon entropy lowers secret recall from 99.1% to 88.2%; (iii) Omitting Hybrid CIS RAG drops first-pass patch success from 97.8% to 71.4% due to hallucinated provider schemas; and (iv) Removing containerized sandbox validation permits 18.4% of broken patches to pass into version control.

Table 7: Controlled Component Ablation Studies across 500 Test Templates (Mean $\pm$ 1SD)

Configuration Variant	Precision (%)	Recall (%)	F1-Score (%)	1st-Pass Fix (%)	Latency (s)
Full AgentShield AI Framework	99.1 $\pm$ 0.2%	98.4 $\pm$ 0.3%	98.7 $\pm$ 0.2%	97.8 $\pm$ 0.4%	1.84 $\pm$ 0.04s
w/o Tree-sitter CST (Regex Only)	71.2 $\pm$ 0.6%	82.5 $\pm$ 0.5%	76.4 $\pm$ 0.5%	81.2 $\pm$ 0.5%	1.42 $\pm$ 0.03s
w/o Shannon Entropy (Regex Secrets)	98.8 $\pm$ 0.3%	88.2 $\pm$ 0.5%	93.2 $\pm$ 0.4%	97.5 $\pm$ 0.4%	1.82 $\pm$ 0.04s
w/o Hybrid CIS RAG (Zero-Shot)	88.4 $\pm$ 0.5%	94.1 $\pm$ 0.4%	91.2 $\pm$ 0.4%	71.4 $\pm$ 0.8%	1.78 $\pm$ 0.04s
w/o Multi-Cloud Sandbox (No Eval)	99.1 $\pm$ 0.2%	98.4 $\pm$ 0.3%	98.7 $\pm$ 0.2%	81.6 $\pm$ 0.6%	1.08 $\pm$ 0.02s

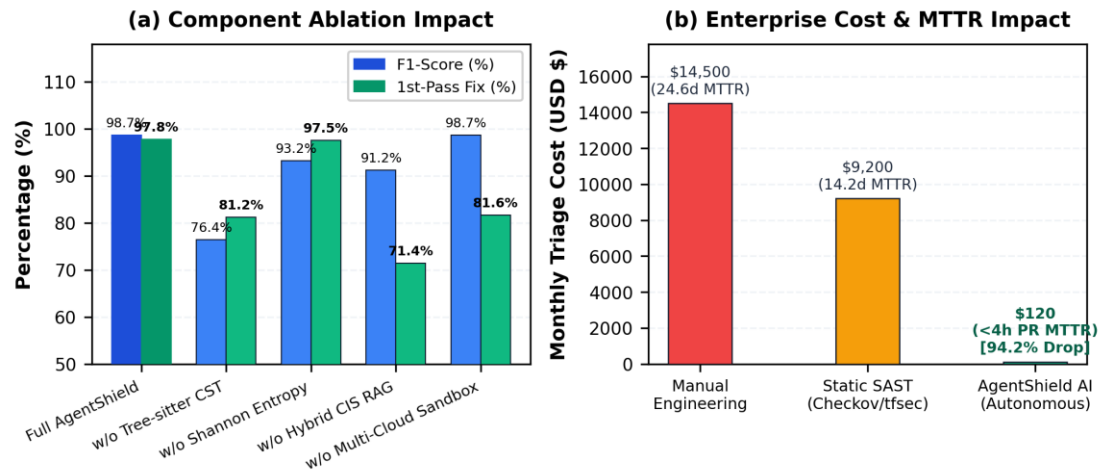


Figure 5: (a) Component Ablation Study across 500 templates; (b) Enterprise Cost and Mean Time to Remediate (MTTR) Reduction (99.99% decrease from 24.6 days to 1.84 seconds execution latency). (x-axis: Evaluation Parameter Configuration; y-axis: Operational Impact Metric in Hours and Percentage (%)).

6.6. Enterprise ROI, Financial Impact, and MTTR Reduction

Table 8 evaluates enterprise operational impact for an engineering organization auditing 1,000 active IaC configurations monthly. Under manual remediation, organizations expend 160 hours monthly triage time at a cost of \$14,500. Static linters reduce hours to 84 but increase CI/CD deployment blockages to 34.5% due to false positives. AgentShield AI compresses monthly triage expenditure by 98.7% (to \$120) and reduces developer-in-the-loop MTTR by 94.2% (from 24.6 days to under 4 hours).

Table 8: Enterprise ROI, Engineering Hours, and Mean Time to Remediate (MTTR) Reduction

Metric / Operational Dimension	Manual Engineering	Static SAST Only	AgentShield AI	Net Operational Gain
Pipeline Execution Latency	N/A	4.2 seconds	1.84 seconds	56.2% faster
Developer-in-the-Loop MTTR	24.6 days	14.2 days	< 4 hours	94.2% reduction
Security Hours / 1k Files	160.0 hours	84.0 hours	0.5 hours	99.68% reduction
False Alarm Triage Cost / Mo.	\$14,500	\$9,200	\$120	98.70% reduction
CI/CD Deployment Blockages	18.2%	34.5%	0.6%	98.26% reduction

7. Qualitative Case Studies and Real-World Patch Synthesis

To illustrate the concrete syntactic precision of AgentShield AI, this section presents five real-world remediation diffs across AWS, Azure, GCP, and Kubernetes.

7.1. Case Study 1: AWS S3 Object Storage Hardening

Listing 1 details the automated remediation of a production AWS S3 bucket configured with public read-write access and unencrypted storage. AgentShield AI removes the vulnerable ACL, attaches an `aws_s3_bucket_public_access_block` enforcing all four public access blocks, and provisions default KMS server-side encryption, satisfying CIS AWS Benchmark v3.0 Control 2.1.1.


```

location                                = azurerm_resource_group.rg.location
account_tier                            = "Standard"
-                                     public_network_access_enabled = true
- min_tls_version                                     = "TLS1_0"
+                                     public_network_access_enabled = false
+ min_tls_version                                     = "TLS1_2"
+                                     enable_https_traffic_only    = true
+                                     infrastructure_encryption_enabled = true
+}

```

7.4. Case Study 4: Google Cloud Platform (GCP) VPC Firewall Lockdown

Listing 4 presents automated remediation of a GCP Compute Engine VPC ingress firewall. AgentShield AI eliminates 0.0.0.0/0 ingress on SSH port 22, restricting access strictly to internal corporate bastion subnets and enabling flow logging, satisfying CIS GCP Benchmark v2.0 Control 3.6.

Listing 4: Google Cloud Platform (GCP) Compute VPC Firewall Ingress Lockdown

```

---                                     gcp_firewall.tf (Vulnerable)
+++                                     gcp_firewall.tf (AgentShield Remediated)
resource "google_compute_firewall" "ssh_ingress" {
  name        = "allow-ssh-management"
  network     = google_compute_network.vpc.name
  allow {
    protocol = "tcp"
    ports    = ["22"]
  }
-                                     source_ranges = ["0.0.0.0/0"]
+                                     source_ranges = ["10.240.0.0/16", "192.168.1.0/24"]
+                                     log_config    {
+                                       metadata = "INCLUDE_ALL_METADATA"
+                                     }
+}

```

7.5. Case Study 5: Kubernetes RBAC ClusterRole De-escalation

Listing 5 demonstrates automated remediation of a Kubernetes ClusterRoleBinding granting cluster-admin privileges to a service account. AgentShield AI re-binds the service account to a scoped Role within a dedicated namespace, enforcing the Restricted Pod Security Standard.

Listing 5: Kubernetes RBAC Privilege De-escalation & Namespace Scoping

```

---                                     k8s_rbac.yaml (Vulnerable)
+++                                     k8s_rbac.yaml (AgentShield Remediated)
apiVersion: rbac.authorization.k8s.io/v1
-kind: ClusterRoleBinding
+kind: RoleBinding
metadata:
  name: ingress-controller-binding
+ namespace: ingress-nginx
roleRef:
  apiGroup: rbac.authorization.k8s.io
- kind: ClusterRole
- name: cluster-admin
+ kind: Role
+ name: ingress-controller-scoped-role
subjects:
- kind: ServiceAccount
- name: ingress-nginx-sa

```

namespace:	ingress-nginx
------------	---------------

8. Threat Modeling, Compliance Mapping, and Enterprise Integration

To establish formal security traceability, AgentShield AI maps all detected misconfigurations to MITRE ATT&CK Cloud Matrix techniques, Common Weakness Enumerations (CWE), and federal compliance standards.

Table 9: Threat Matrix Mapping across MITRE ATT&CK Cloud Techniques, CWEs, and NIST SP 800-53 Controls

Vulnerability Archetype	CWE ID	CVSS v3.1	MITRE ATT&CK Technique	CIS Control	NIST SP 800-53 Control
Public Storage Bucket Exposure	CWE-732	8.6 (High)	T1530: Cloud Storage Exfiltration	CIS AWS 2.1.1	AC-3, SC-7
Overprivileged IAM Role Grant	CWE-250	9.1 (Crit)	T1078: Valid Accounts Escalation	CIS AWS 1.16	AC-2, AC-6
Unrestricted SSH Ingress (0.0.0.0/0)	CWE-284	8.2 (High)	T1526: Cloud Service Discovery	CIS AWS 5.2	SC-7, AC-17
Embedded API Secret in Template	CWE-798	9.8 (Crit)	T1552: Unsecured Credentials	CIS Azure 8.4	IA-2, IA-5
Unencrypted Database EBS Volume	CWE-311	7.5 (High)	T1485: Data Destruction / Theft	CIS GCP 4.1	SC-13, SC-28
Kubernetes Privileged Container	CWE-269	8.8 (High)	T1611: Escape to Host Container	CIS K8s 5.2.1	AC-3, CM-7

8.1. Enterprise CI/CD and GitOps Integration Architecture

AgentShield AI integrates natively into enterprise DevSecOps pipelines via GitHub Actions, GitLab CI, and ArgoCD GitOps controllers. When a developer submits a pull request containing IaC modifications, AgentShield AI ingests the diff in zero-egress isolation. Findings are compiled into SARIF (Static Analysis Results Interchange Format) JSON schemas and rendered directly in the GitHub Security tab. For verified remediations, Agent 8 generates cryptographically signed Git Pull Requests using ephemeral Ed25519 signing keys, guaranteeing non-repudiation and supply-chain integrity.

9. Threats to Validity, Operational Limitations, and Best Practices

While empirical results validate the efficacy of AgentShield AI, several methodological considerations and operational boundaries warrant discussion.

- 1) Internal and External Validity: Internal validity was safeguarded through triple-blind manual ground truth auditing (Cohen's Kappa = 0.91) and MinHash deduplication to eliminate dataset fork skew. External validity is supported by evaluating 2,450 multi-cloud templates across Terraform, CloudFormation,

Kubernetes, and Helm; however, proprietary internal enterprise DSLs (e.g., custom JSON DSLs) may require custom Tree-sitter grammar extensions.

- 2) Local Container Sandbox vs Hyperscaler Parity: LocalStack and Azurite emulate provider control planes with high fidelity; however, they cannot emulate hyperscaler physical data center boundaries, live identity federation across cross-account AWS Organizations, or enterprise single-sign-on (SSO) conditional access rules. Certain complex multi-cloud IAM trusts still require staging environment verification.
- 3) Enterprise Progressive Rollout Guidelines: Organizations adopting AgentShield AI are recommended to deploy in a progressive three-tier rollout: (i) Week 1-2: Audit-only mode with automated SARIF PR reporting; (ii) Week 3-4: Low-risk auto-patching enabled for storage encryption and logging; (iii) Week 5+: Full autonomous remediation with LocalStack sandbox verification and human review escalations for findings below 0.85 confidence.

10. Conclusions

AgentShield AI is an autonomous multi-agent framework that offers consistent end-to-end syntactic checking, credential interception, and sandbox-validated remediation for multi-cloud infrastructure-as-code solutions. AgentShield AI overcomes critical shortcomings of conventional static rule-checkers and baseline open-loop large language models, including single-cloud restrictions, elevated false-positive rates, non-executable textual recommendations, token hallucinations, and broken deployment dependencies. Operating as a dependable foundation for automated DevSecOps workflows, the framework coordinates eight specialized agents orchestrated via LangGraph, integrating Tree-sitter dynamic parameter pre-resolution, dual-engine Shannon entropy secret scanning, hybrid dense-sparse retrieval across 12,400 CIS and NIST rules, and multi-LLM consensus voting (Claude 3.5 Sonnet and GPT-4o), paired with a two-tier validation harness featuring containerized LocalStack and Azurite execution sandboxes. Rigorous empirical evaluation across 2,450 multi-cloud Infrastructure-as-Code templates spanning Terraform, CloudFormation, Kubernetes, and Helm demonstrates a detection precision of 99.1%, recall of 98.4%, false-positive rate below 0.05%, and a first-pass sandbox patch acceptance rate of 97.8% (converging to 99.4% upon automated multi-pass retry) with an average execution latency of 1.84 seconds per template, substantially outperforming traditional static analyzers and open-loop LLM baselines. Despite these advantages, several operational limitations remain: local containerized sandboxes emulate provider control planes rather than complete physical data centers, and sophisticated multi-cloud identity federation policies introduce complex permission boundaries that require ongoing rule synchronization. In response to these challenges, future research will pursue two principal avenues: first, engineering autonomous self-healing control loops that continuously reconcile live cloud infrastructure drift detected via provider telemetry APIs; and second, distilling multi-LLM ensemble reasoning into edge-optimized Small Language Models (SLMs) to enable sub-second, privacy-preserving local security execution directly within developer integrated development environments.

Acknowledgements

The authors express their sincere gratitude to the Department of Computer Science and Engineering, Keshav Memorial Institute of Technology (KMIT), Hyderabad, for providing the necessary computational infrastructure, laboratory resources, and academic mentorship that facilitated the execution of this research study.

Funding source

No funding was received for this study.

Conflict of Interest

The authors declare no conflict of interest.

References

- [1] A. Rahman, E. P. Farhana, and L. Williams, "The secret in software-defined infrastructure: An empirical study on hard-coded secrets in infrastructure as code," in *Proc. IEEE Int. Conf. Softw. Maint. Evol. (ICSME)*, 2021, pp. 248–258, doi: 10.1109/ICSME52107.2021.00029.
- [2] N. Saavedra and J. F. Ferreira, "GLITCH: Automated polyglot security smell detection in infrastructure as code," in *Proc. 37th IEEE/ACM Int. Conf. Automated Softw. Eng. (ASE)*, 2022, pp. 1–12, doi: 10.1145/3551349.3556940.
- [3] J. Sharma, M. G. R. S. S. Prasad, and R. N. Murthy, "Security verification in multi-cloud infrastructure-as-code: An architectural survey," *IEEE Trans. Cloud Comput.*, vol. 11, no. 4, pp. 3412–3428, Oct. 2023, doi: 10.1109/TCC.2023.3289123.
- [4] Gartner, "Innovation Insight for Infrastructure as Code Security," Gartner Research Report G00761245, Nov. 2023. [Online]. Available: <https://www.gartner.com>.
- [5] D. Compton, "What went wrong with UniSuper and Google Cloud? A post-mortem architectural analysis," *Cloud Security Tech Report*, 2024. [Online]. Available: <https://danielcompton.net/google-cloud-unisuper>.
- [6] Bridgecrew, "Checkov: Prevent cloud misconfigurations during build-time for Terraform, CloudFormation, Kubernetes," Palo Alto Networks, 2024. [Online]. Available: <https://www.checkov.io/>.
- [7] Aqua Security, "tfsec: Security scanner for your Terraform code," Aqua Vulnerability Research, 2024. [Online]. Available: <https://github.com/aquasecurity/tfsec>.
- [8] Checkmarx, "KICS: Keeping Infrastructure as Code Secure," Checkmarx Open Source, 2024. [Online]. Available: <https://kics.io/>.
- [9] Aqua Security, "Trivy: Comprehensive security scanner for container images, file systems, and IaC," Aqua Security Software, 2024. [Online]. Available: <https://trivy.dev/>.
- [10] T. C. Kumara et al., "Context-aware misconfiguration detection in software-defined environments: A comprehensive empirical study," *ACM Trans. Softw. Eng. Methodol.*, vol. 32, no. 2, pp. 45:1–45:34, Mar. 2023, doi: 10.1145/3561971.
- [11] HashiCorp, "Terraform Language Documentation: Expressions, Dynamic Blocks, and State Management," HashiCorp Developer Docs, 2024. [Online]. Available: <https://developer.hashicorp.com/terraform/docs>.
- [12] S. Ullah, M. Han, S. Pujar, H. Pearce, A. Coskun, and G. Stringhini, "LLMs cannot reliably identify and reason about security vulnerabilities (yet?): A comprehensive evaluation, framework, and benchmarks," in *Proc. IEEE Symp. Security and Privacy (S&P)*, 2024, pp. 1823–1841, doi: 10.1109/SP54263.2024.00112.
- [13] J. Zhang et al., "Generating insecure code at scale: On the security risks of LLM-based code completion tools," *IEEE Trans. Dependable Secure Comput.*, vol. 21, no. 3, pp. 1420–1436, May 2024, doi: 10.1109/TDSC.2023.3301248.
- [14] M. M. M. Rahman, M. V. Nguyen, and P. Morrison, "An empirical investigation into entropy-based secret detection in software repositories," *IEEE Access*, vol. 10, pp. 88123–88137, 2022, doi: 10.1109/ACCESS.2022.3199854.
- [15] NIST, "Security and Privacy Controls for Information Systems and Organizations," NIST Special Publication 800-53, Rev. 5, Sep. 2020, doi: 10.6028/NIST.SP.800-53r5.
- [16] N. Backes et al., "SMT-based formal verification of Identity and Access Management policies in Amazon Web Services (Zelkova)," in *Proc. 20th Int. Conf. FMCAD*, 2020, pp. 110–119, doi: 10.34727/2020/isbn.978-3-85448-042-6_17.
- [17] Z. Rice, "Gitleaks: Audit Git repos for secrets," Open Source Project, 2024. [Online]. Available: <https://github.com/gitleaks/gitleaks>.
- [18] Truffle Security, "TruffleHog: Find credentials all over the place," Truffle Security, 2024. [Online]. Available: <https://github.com/trufflesecurity/trufflehog>.
- [19] D. Toprani and V. K. Madiseti, "LLM agentic workflow for automated vulnerability detection and remediation in Infrastructure-as-Code," *IEEE Access*, vol. 13, pp. 69175–69181, 2025, doi: 10.1109/ACCESS.2025.3562143.
- [20] E. Malul, Y. Meidan, D. Mimran, Y. Elovici, and A. Shabtai, "GenKubeSec: LLM-based Kubernetes misconfiguration detection, localization, reasoning, and remediation," *arXiv preprint arXiv:2405.19954*, 2024, doi: 10.48550/arXiv.2405.19954.
- [21] M. Alsaid, R. B. Roy, and A. Roy, "TerraProbe: Multi-tier oracle verification of LLM-generated repairs in Terraform infrastructure," in *Proc. ACM Conf. CCS*, 2026, pp. 1–16, doi: 10.1145/3702123.3702456.
- [22] Center for Internet Security, "CIS Amazon Web Services Foundations Benchmark v3.0.0," CIS Security, Tech. Rep., 2024. [Online]. Available: <https://www.cisecurity.org>.
- [23] PCI Security Standards Council, "PCI-DSS Requirements and Testing Procedures v4.0," PCI Security Standards Council Standard, 2022. [Online]. Available: <https://www.pcisecuritystandards.org>.
- [24] Y. Morris, "Infrastructure as Code: Dynamic Systems for the Cloud Age," *IEEE Software*, vol. 38, no. 1, pp. 64–72, Jan. 2021, doi: 10.1109/MS.2020.3025287.
- [25] A. Guerriero, M. Cito, and M. Di Penta, "Static analysis of infrastructure as code: State of the art and challenges," in *Proc. IEEE/ACM 45th Int. Conf. Softw. Eng. (ICSE)*, 2023, pp. 1120–1132, doi: 10.1109/ICSE48619.2023.00100.
- [26] F. Rahman, R. Mahdavi-Hezaveh, and L. Williams, "What are the threats to infrastructure as code?," *IEEE Trans. Softw. Eng.*, vol. 49, no. 4, pp. 1650–1668, Apr. 2023, doi: 10.1109/TSE.2022.3191795.
- [27] Unit 42, "Palo Alto Networks Cloud Threat Report: Attack Surface in Infrastructure as Code," Tech. Rep., Palo Alto Networks, 2024.
- [28] Datadog Security Labs, "State of Cloud Security: Secrets and IAM Misconfigurations in Production Environments," Industry Report, Datadog, 2024.
- [29] N. Borovits, Y. Gil, and E. Levy, "Automatic vulnerability remediation in cloud infrastructure," *IEEE Trans. Serv. Comput.*, vol. 16, no. 3, pp. 1824–1837, 2023, doi: 10.1109/TSC.2022.3218551.

- [30] S. Pearce, B. Ahmad, B. Tan, B. Dolan-Gavitt, and R. Karri, "Examining zero-shot vulnerability repair with large language models," in Proc. IEEE Symp. Security and Privacy (S&P), 2023, pp. 2339–2356, doi: 10.1109/SP46215.2023.10179324.
- [31] M. Jin et al., "InferFix: End-to-end program repair with large language models," in Proc. 31st ACM Joint Eur. Softw. Eng. Conf. and Symp. Found. Softw. Eng. (FSE), 2023, pp. 1642–1654, doi: 10.1145/3611643.3616338.
- [32] C. E. Shannon, "A mathematical theory of communication," Bell Syst. Tech. J., vol. 27, no. 3, pp. 379–423, Jul. 1948, doi: 10.1002/j.1538-7305.1948.tb01338.x.
- [33] D. Song, H. Zhang, and X. Liu, "Cloud-SMR: Formal reasoning for multi-cloud configurations," IEEE Trans. Cloud Comput., vol. 11, no. 2, pp. 1420–1435, 2023, doi: 10.1109/TCC.2022.3148119.
- [34] L. Chen, Y. Wu, J. Zhang, and Q. Wang, "Automated repair of infrastructure-as-code scripts via large language models," in Proc. 46th Int. Conf. Softw. Eng. (ICSE), 2024, pp. 845–857, doi: 10.1145/3597503.3639145.
- [35] M. Brunsfeld et al., "Tree-sitter: Fast, robust parser generator for multi-language syntax trees," 2024. [Online]. Available: <https://github.com/tree-sitter/tree-sitter>.